

Step 1 — Wallet Connection & Authentication (EIP-191, EIP-1193)

This step happens before any smart contract is touched. It's entirely off-chain — no gas spent, no transaction broadcast — but it's what establishes *who* is about to act, so every later step depends on it.

The two EIPs in play here

- **EIP-1193** — the standardized browser-wallet provider interface. When the frontend calls `window.ethereum.request({ method: 'eth_requestAccounts' })`, this EIP is *why* that call works identically regardless of which wallet extension is installed. It never touches a smart contract; it's a JavaScript-to-wallet-extension interface, not a blockchain interface.
- **EIP-191** — the "personal sign" message format. When Alice's wallet signs the login message, it doesn't sign the raw bytes — it computes `keccak256("\x19Ethereum Signed Message:\n" + len(message) + message)` and signs *that* hash. This specific prefix is what makes a wallet show Alice a readable string to approve, instead of a meaningless hex blob, and it's also what prevents this signature from being replayable as if it were a signature over a raw transaction or a different kind of structured message (EIP-712, covered in Step 4, uses a different prefix for exactly this reason — the two schemes are deliberately non-interchangeable).

The actual sequence

1. Frontend calls `GET /auth/nonce?address=0x1111111111111111111111111111111111111111AAAA`

Input: Alice's address as a query parameter.

Backend (`app/auth.py` , `generate_nonce`):

- Generates `nonce = secrets.token_hex(16)` — a random 32-character hex string, e.g. `"7f3a9c21e8b4d605f19a2c8e4b7d1f30"`.
- Computes `expires_at = now + 5 minutes`.
- Stores `WalletNonce(address="0x1111...aaaa", nonce="7f3a9c21...", expires_at=..., used=False)`.
- Calls `build_login_message(nonce)`, which returns exactly: `"Aurelia Labs login nonce: 7f3a9c21e8b4d605f19a2c8e4b7d1f30"`

Output: `{ address: "0x1111...aaaa", nonce: "7f3a9c21...", message: "Aurelia Labs login nonce: 7f3a9c21..." }`

Why the backend computes and returns the exact message string, rather than the frontend constructing it: if two independent pieces of code each tried to build this string, a single whitespace or casing difference between them would make every signature fail verification with no obvious cause. One source of truth, handed to the frontend, removes that entire failure class.

2. Alice's wallet signs the message

Input: the exact string `"Aurelia Labs login nonce: 7f3a9c21e8b4d605f19a2c8e4b7d1f30"`.

Traversal: the wallet extension applies the EIP-191 prefix, hashes it, and signs the hash with Alice's private key (never transmitted or exposed — this happens inside the wallet, MetaMask or otherwise).

Output: a 65-byte signature, e.g. `0xa1b2c3...` (r: 32 bytes, s: 32 bytes, v: 1 byte) — the exact hex value is unique to Alice's key and this specific message; even a single changed character in the message would produce a signature that recovers to a completely different (wrong) address.

3. Frontend calls POST /auth/wallet with { address, signature }

Backend (app/auth.py , verify_signature_and_consume_nonce):

- [illegible]

Output on success: a signed JWT, `create_access_token("0x1111...aaaa")` — payload `{ sub: "0x1111...aaaa", iat: <now>, exp: <now + 24h> }`, signed with the backend's own `SECRET_KEY`. This token is what every subsequent HTTP call (Steps 2, 3, 5) attaches as `Authorization: Bearer <token>`.

What this step does *not* do

No smart contract is called. No gas is spent. `InsurancePolicy`, `ClaimRegistry`, and every other contract are completely unaware this happened — as far as the blockchain is concerned, Alice doesn't exist yet until Step 2's transaction actually lands. This step only establishes an authenticated HTTP session with the backend.